

Documento Técnico: Flujos Transaccionales — Proyecto HeartCoin

Introducción

Este documento describe y analiza en detalle los flujos transaccionales del proyecto HeartCoin, entendidos como aquellas secuencias de operaciones que modifican el estado del sistema de forma significativa —particularmente el saldo de HeartCoins (HC) de un usuario— e involucran validaciones de negocio, múltiples componentes del sistema, o comunicación con servicios externos. Se documenta cada flujo con el detalle de sus pasos, sus validaciones, sus condiciones de error y los componentes técnicos involucrados.

Objetivo

Documentar de forma completa y verificable cada flujo transaccional relevante del sistema, explicando la secuencia exacta de operaciones, los puntos de validación, los mecanismos de consistencia aplicados, y las condiciones bajo las cuales cada flujo puede fallar o revertirse.

Alcance

Este documento cubre los flujos que afectan el ledger de HeartCoin (`hc_transactions`), los flujos de identidad con impacto transaccional (registro con verificación de correo, eliminación de cuenta), y los flujos de integración con terceros que involucran movimiento de valor (Original Pay). No cubre flujos de interacción social sin impacto económico (publicar, comentar, seguir), documentados en `ARQUITECTURA_TECNICA.md`.

Metodología

Los flujos aquí descritos se reconstruyeron a partir de la lectura directa del código fuente de las funciones `SECURITY_DEFINER` y `Edge Functions` involucradas, y fueron **verificados mediante ejecución real** contra el entorno de producción (no solo inspección estática), incluyendo pruebas de los casos de éxito, saldo insuficiente, firma inválida y reembolso por rechazo.

1. Clasificación de Flujos Transaccionales

Los flujos del sistema se agrupan en cuatro categorías:

1. **Flujos de acreditación de HC:** el usuario gana HC por una actividad válida.
2. **Flujos de consumo de HC:** el usuario gasta HC a cambio de un beneficio, servicio o pago.

3. **Flujos de integración externa con valor monetario:** Original Pay consulta o mueve HC en nombre de un usuario.
4. **Flujos transaccionales de identidad:** registro y eliminación de cuenta, que afectan de forma atómica y en cascada la totalidad del estado del usuario.

2. Flujo: Acreditación de HC por Check-in en Iniciativa

Disparador: el usuario escanea el código QR de una iniciativa presencial.

```
1. App captura la ubicación GPS del dispositivo 2. App invoca la función RPC
checkin_iniciativa(iniciativa_id, lat, lng) 3. La función valida, en el servidor: a. Que
el método de verificación de la iniciativa sea 'qr' b. Que la hora actual esté dentro de
la ventana permitida (event_date + verification_window_hours) c. Que la distancia
(fórmula haversine) entre la ubicación reportada y la ubicación registrada de la
iniciativa sea  $\leq 200$  metros 4. Si todas las validaciones pasan, inserta una fila en
iniciativa_checkins (clave primaria compuesta iniciativa_id + user_id, evita duplicados)
5. Trigger grant_hc_for_checkin se dispara automáticamente: inserta en hc_transactions
(type: 'checkin', amount: +hc_reward) 6. Trigger apply_hc_transaction sincroniza
personal_profiles.hc_balance 7. Un certificado de participación se genera a partir de
este check-in
```

Condiciones de fallo: método de verificación distinto a qr, fuera de ventana horaria, fuera de rango de distancia, check-in duplicado (violación de clave primaria). Todas las validaciones ocurren en el servidor; el cliente no puede omitirlas incluso si estuviera modificado.

3. Flujo: Acreditación de HC por Voto en Votación

```
1. Usuario emite su voto de apoyo en una votación de campaña 2. Inserción en
votacion_votes (restricción única votacion_id + user_id, impide votar dos veces) 3.
Trigger grant_hc_for_votacion_vote se dispara automáticamente: inserta en
hc_transactions (type: 'voto', amount: +hc_reward) 4. Trigger apply_hc_transaction
sincroniza hc_balance
```

Este flujo es más simple que el check-in al no requerir validación de geolocalización ni ventana temporal — refleja una decisión de diseño donde votar/apoyar se considera una actividad de menor esfuerzo que un check-in presencial.

4. Flujo: Canje de Beneficio con HC

```
1. Usuario escanea el código QR de un beneficio publicado por una empresa 2. App invoca
la función RPC redeem_beneficio(beneficio_id) 3. La función valida, en el servidor (con
bloqueo de fila FOR UPDATE sobre el beneficio): a. Que el beneficio exista y esté en
```

estado 'activo' b. Que no se haya alcanzado `max_redemptions` c. Que el usuario no lo haya canjeado previamente 4. Si el tipo de beneficio es 'cashback': inserta en `hc_transactions` (type: 'redemption_cashback', amount: +`hc_reward`) Si es descuento/beca: valida que `hc_balance` $\geq$ `hc_cost`, luego inserta (type: 'redemption_spend', amount: -`hc_cost`) 5. Inserta en `beneficio_redemptions` (clave compuesta, evita doble canje) 6. Actualiza el contador `redemptions_count` del beneficio 7. Trigger `apply_hc_transaction` sincroniza `hc_balance`

Nota de diseño: el bloqueo de fila (FOR UPDATE) se aplica sobre el registro del **beneficio**, protegiendo el límite global de canjes (`max_redemptions`) ante concurrencia. No existe un bloqueo equivalente sobre la fila del **perfil del usuario** al verificar `hc_balance`, lo cual se documenta como área de mejora en la sección de riesgos.

5. Flujo: Canje de Servicio con HC

Estructuralmente idéntico al flujo anterior, ejecutado por la función `redeem_servicio(servicio_id)` sobre las tablas `servicios/servicio_redemptions`, utilizando `pricing_type` (costo/cashback) en lugar de `benefit_type`.

6. Flujo: Consulta de Saldo desde Original Pay

1. Original Pay firma una petición GET con HMAC-SHA256 (API key + timestamp + nonce + hash del cuerpo) 2. Edge Function `originalpay-balance` recibe la petición 3. Verifica la firma; si no coincide o el timestamp excede ± 5 minutos, responde 401 sin consultar la base de datos 4. Busca en `personal_profiles` por email 5. Responde con { email, `hc_balance` } o 404 si no existe el usuario

Este flujo es de solo lectura y no genera ninguna fila en `hc_transactions`.

7. Flujo: Cobro con HC desde Original Pay

1. Original Pay firma una petición POST con HMAC hacia `originalpay-redeem`, incluyendo email, `amount_hc` y `order_reference` 2. La Edge Function verifica la firma HMAC 3. Busca al usuario por email; si no existe, responde 404 4. Verifica que `hc_balance` $\geq$ `amount_hc`; si no, responde 409 (`hc_insuficiente`) 5. Inserta en `hc_transactions`: (type: 'redemption_spend', reference_type: 'originalpay', external_reference: `order_reference`, amount: -`amount_hc`) 6. Trigger `apply_hc_transaction` sincroniza `hc_balance` 7. Responde con { `transaction_id`, `new_balance` }

Este flujo fue **verificado mediante ejecución real**: se acreditó saldo de prueba, se ejecutó un cobro exitoso, se confirmó la actualización del saldo, y se limpiaron los datos de prueba posteriormente.

8. Flujo: Reembolso Automático por Webhook de Original Pay

Este es el flujo transaccional de mayor complejidad condicional del sistema, dado que reacciona a un evento externo asíncrono que puede ocurrir en cualquier momento después de que el cobro original ya fue procesado.

```
1. Original Pay notifica el resultado final de un pago vía POST a originalpay-webhook, incluyendo token_app, secret_key, order_number y payment_status ('approved' o 'rejected') 2. Se verifican token_app/secret_key contra los valores esperados (comparación de tiempo constante); si no coinciden, responde 401 3. Si payment_status = 'approved': no se ejecuta ninguna acción sobre el ledger (el HC ya se descontó en el paso 7 anterior) – solo se reconoce la notificación con 200 4. Si payment_status = 'rejected': a. Se busca en hc_transactions la transacción original por reference_type = 'originalpay' + external_reference = order_number b. Se verifica que no exista ya un reembolso previo para ese order_number (reference_type = 'originalpay_refund') – control de idempotencia ante reintentos del webhook c. Si no hay reembolso previo, se inserta una transacción compensatoria: (type: 'ajuste', reference_type: 'originalpay_refund', amount: +|monto original|) d. Trigger apply_hc_transaction restaura el saldo del usuario 5. Se responde 200 { received: true } en todos los casos válidos
```

Verificación realizada: se ejecutó el flujo completo de extremo a extremo (cobro real → webhook de rechazo → confirmación de que el saldo se restauró exactamente al valor original → reenvío del mismo webhook → confirmación de que el reembolso no se duplicó).

Decisión de diseño relevante: el monto a reembolsar se calcula a partir del **propio ledger de HeartCoin** (el monto de la transacción original), no a partir del campo `amount/currency` que envía el webhook (expresado en la moneda de curso legal de Original Pay, no en HC), evitando así cualquier dependencia de una tasa de conversión externa no controlada por HeartCoin.

9. Flujo: Registro de Cuenta con Verificación de Correo

```
1. Usuario completa el formulario de registro en la app 2. Se crea la cuenta en Supabase Auth y la fila de perfil correspondiente (personal_profiles / organization_profiles / company_profiles) 3. App invoca la Edge Function originalauth-register con la contraseña elegida por el usuario 4. La función valida la sesión recién creada, y llama a Original Auth: POST /api/v1/integrations/registration (firmado con HMAC), con el mismo correo y contraseña 5. Si Original Auth confirma el registro, se guarda su identificador (original_auth_id) en el perfil de HeartCoin 6. Original Auth envía un correo con código de verificación 7. App bloquea el acceso a la aplicación y muestra la pantalla de verificación 8. Usuario ingresa el código; app invoca originalauth-verify-email 9. Si el código es válido, se marca email_verified = true en el perfil 10. Usuario accede normalmente a la aplicación (vía sesión de Supabase, sin relación con Original Auth a partir de este punto)
```

Este flujo no mueve HC, pero es transaccional en el sentido de que involucra múltiples sistemas (Supabase + Original Auth) que deben coordinarse de forma consistente; un fallo en el paso 4 no revierte

la cuenta ya creada en el paso 2, condición documentada como pendiente de definición de producto (ver sección de riesgos).

10. Flujo: Eliminación de Cuenta

1. Usuario confirma la eliminación desde la app (checkbox obligatorio) 2. App invoca la Edge Function delete-account 3. La función valida la sesión del usuario mediante su token 4. Con la clave de servicio de Supabase, ejecuta `auth.admin.deleteUser(userId)` 5. La eliminación de la fila en `auth.users` dispara, en cascada (`ON DELETE CASCADE`), la eliminación de la totalidad de los datos asociados al usuario en las 30 relaciones de clave foránea del esquema: perfil, publicaciones, comentarios, likes, seguimiento, bloqueos, historial de HC, canjes, votos, check-ins, notificaciones

Este es el flujo transaccional de mayor alcance del sistema en términos de volumen de datos afectados, y a la vez el de menor complejidad de implementación, gracias al diseño consistente de integridad referencial descrito en `MODELO_DE_DATOS.md`.

Problemas Detectados

1. **Ausencia de atomicidad estricta en la verificación de saldo** (flujos 4, 5 y 7): la lectura del saldo disponible y la inserción de la transacción de descuento no ocurren dentro de una única operación atómica a nivel de base de datos en todos los casos (parcialmente mitigado por bloqueo de fila en el caso de `redeem_beneficio/redeem_servicio` sobre la entidad canjeada, no sobre el perfil del usuario).
2. **El flujo de registro (sección 9) no es transaccional entre sistemas**: si la llamada a Original Auth falla después de que la cuenta ya se creó en Supabase, el usuario queda con una cuenta funcional pero sin proceso de verificación iniciado, sin un mecanismo de reintento automático más allá del reenvío manual de verificación.
3. **Dependencia de la entrega de correo de un tercero**: se identificó, durante la verificación de flujos, que la entrega real del correo de verificación (Original Auth, ambiente de producción) no pudo confirmarse de forma consistente, bloqueando la finalización del flujo 9 para las cuentas afectadas. Esta es una dependencia externa fuera del control directo de HeartCoin.

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto financiero-operativo bajo** dado el volumen actual del sistema, pero debe reevaluarse ante un incremento significativo de transacciones concurrentes, particularmente en el flujo 7 (iniciado por un tercero externo cuyo volumen no está bajo control directo de HeartCoin).
- El riesgo del punto 2 puede derivar en **cuentas en estado intermedio permanente** (creadas pero nunca verificadas), lo cual no bloquea el uso de la aplicación bajo el diseño actual (la verificación no se exige en el login), pero sí impide completar cualquier proceso de negocio que dependa de una

verificación de correo confirmada.

- El riesgo del punto 3 es **externo y no mitigable unilateralmente** por HeartCoin; su impacto se limita actualmente al flujo de verificación de correo, pero podría extenderse a cualquier flujo futuro que dependa de la misma vía de entrega.

Fortalezas Identificadas

- **Verificación real de los flujos críticos**, no solo documentación teórica: los flujos de saldo, cobro y reembolso de Original Pay fueron ejecutados y confirmados de extremo a extremo antes de considerarse completos.
- **Idempotencia explícita en el flujo de reembolso** (flujo 8), verificada mediante reenvío deliberado del mismo evento.
- **Cálculo de reembolso basado en la fuente de verdad interna** (el propio ledger), no en datos potencialmente ambiguos de un sistema externo.
- **Consistencia del patrón transaccional** a través de los distintos flujos de acreditación y consumo de HC, todos convergiendo en el mismo mecanismo de ledger + trigger, facilitando su auditoría conjunta.

Recomendaciones

1. Extender el uso de bloqueo de fila (`FOR UPDATE`) a la verificación de `hc_balance` en los flujos de consumo (4, 5 y 7), no únicamente a la entidad canjeada, para eliminar la condición de carrera residual identificada.
2. Definir un procedimiento operativo (manual o automatizado) para identificar y dar seguimiento a cuentas que completaron el paso 2 del flujo de registro sin completar el paso 9 (verificación de correo pendiente por tiempo prolongado).
3. Evaluar, junto con el proveedor de Original Auth, un mecanismo de confirmación de entrega de correo (o un canal de verificación alternativo) que no dependa exclusivamente de la recepción exitosa del correo electrónico.

Conclusión

Los flujos transaccionales de HeartCoin, tanto los internos (acreditación y consumo de HC) como los que involucran integraciones externas (Original Auth, Original Pay), siguen un patrón consistente de validación en el servidor, registro inmutable en el ledger, y sincronización automática del estado derivado. La verificación activa realizada sobre los flujos más críticos —particularmente el ciclo completo de cobro y reembolso con Original Pay— proporciona un nivel de confianza superior al de una revisión puramente documental. Las oportunidades de mejora identificadas son de naturaleza incremental y no comprometen

la solidez general del diseño transaccional del sistema.

Anexo: Tabla Resumen de Flujos

Flujo	Dispara acreditación/consumo de HC	Involucra terceros externos	Verificado mediante ejecución real
Check-in en iniciativa	Acreditación	No	Sí (funcionalidad en producción)
Voto en votación	Acreditación	No	Sí (funcionalidad en producción)
Canje de beneficio	Consumo o acreditación (cashback)	No	Sí (funcionalidad en producción)
Canje de servicio	Consumo o acreditación (cashback)	No	Sí (funcionalidad en producción)
Consulta de saldo (Original Pay)	Ninguno (solo lectura)	Sí	Sí
Cobro con HC (Original Pay)	Consumo	Sí	Sí
Reembolso por webhook (Original Pay)	Acreditación compensatoria	Sí	Sí
Registro con verificación de correo	Ninguno	Sí (Original Auth)	Sí
Eliminación de cuenta	N/A (elimina el historial completo)	No	Sí (funcionalidad en producción)